

STRIVO CONSULTANCY WEBSITE

PROJECT DOCUMENTATION REPORT

A MERN Stack Consultancy Web Application

TECHNOLOGY STACK

MongoDB Express.js React.js Node.js

SUBMITTED BY: TEAM G

Namitha

Vishnu

Nejuma

PROJECT LINKS

Deployed Link: <https://strivo-psi.vercel.app/>

GitHub Repository: <https://github.com/Vishnuprasadsv/strivo-consultancy.git>

Date of Submission: June 2026

Table of Contents

1. Project Takeaways	3
2. Project Summary & Description	3
3. Project Scope	3
4. Key Features	4
5. Target Users	4
6. Technical Implementation Details	5
7. Authentication & Authorization	6
8. Responsive Design Approach	6
9. Error Handling	6
10. Security Measures	7
11. Performance Optimizations	7
12. Git Workflow Followed During Development	7
13. System Architecture & Data Flow	8
14. Project Structure	9
15. Website Pages Hub & System Catalog	10
16. Code Modules & API Routes	13
17. Database Design	14
18. Deployment Process	15
19. Challenges Faced During Development	15
20. Future Enhancements	15
21. Conclusion	15

1. Project Takeaways

Working on the Strivo Consultancy Website was a valuable learning experience that went beyond technical development. The project helped us understand the importance of collaboration, effective communication, and teamwork in building a complete full-stack application.

Key Takeaways:

- Working as a team allowed us to share ideas, discuss different approaches, and learn from one another throughout the project.
- Regular communication and progress discussions helped us stay aligned and complete tasks efficiently.
- Collaborating with teammates improved our understanding of both frontend and backend development, as we frequently exchanged knowledge and supported each other when solving technical challenges.
- Listening to different perspectives helped us make better design and implementation decisions while improving the overall quality of the application.
- Dividing responsibilities among team members made the development process more organized and ensured that project milestones were completed on time.
- Resolving issues together during development strengthened our problem-solving skills and taught us the value of collaborative debugging.
- Using Git and GitHub as part of our daily workflow gave us practical experience with feature branches, commits, pull requests, and version control in a team environment.
- The project improved our ability to plan tasks, coordinate development activities, and work effectively as part of a software development team.
- Overall, this project enhanced both our technical knowledge and our confidence in working collaboratively on real-world web development projects.

2. Project Summary & Description

Strivo Consultancy is a full-stack web application designed to manage and streamline consultancy operations. It features a robust backend built with Node.js and Express, connected to a MongoDB database, and a dynamic, responsive frontend built with React, Vite, and Tailwind CSS. The platform includes secure authentication, file management via Cloudinary, email notifications, and an admin dashboard.

3. Project Scope

The scope of this project includes the design, development, and testing of a responsive, full-stack web application consisting of a React client, an Express backend server, and a MongoDB database. The scope covers the frontend UI animations, client form verification, file uploads management (via Cloudinary integration), secure JWT-based admin login authorization, database aggregation for dashboard statistics, and deployment verification. The system is scoped to scale gracefully across mobile, tablet, and widescreen desktop layouts.

4. Key Features

Client-Facing Portal:

- Dynamic Testimonials Swiper: Automatically pulls approved success stories and reviews from the MongoDB database and renders them inside an animated slider.
- Insights & Articles Engine: Categorized articles list featuring pagination, responsive card layouts, and detailed article view pages.
- Interactive Mission & Vision Sections: Dynamic, interactive pages that detail the firm's methodologies with hover effects.
- Inbound Leads Contact Form: Lead capture form with real-time feedback and server-side submission tracking.
- Integrated Career Portal: Displays current job roles and supports direct resume files upload.

Administrative Panel:

- Statistics Dashboard: Real-time counter detailing the number of active articles, submitted inquiries, and resumes.
- Inquiry Manager: Lists, organizes, and processes lead submissions.
- Resume & Applicant Desk: Secure view and download interface for job applications and resume documents.
- Articles Console: Content management panel to create, edit, delete, and publish insights articles.
- Testimonial Approvals Desk: Real-time moderation tool to review user-submitted reviews before publishing.

5. Target Users

The platform supports three distinct target user profiles:

- Corporate Clients: Business owners, CEOs, and corporate managers seeking strategic, operational, or technical consulting partnerships.
- Job Candidates: Professional strategists, data scientists, and engineers looking to join Strivo's team.
- Strivo System Administrators: Corporate managers who require administrative access to manage site content, review leads, and download applicant resumes.

6. Technical Implementation Details

Implementation Framework Matrix

Stack Component	Technologies & Implementation Purpose
Frontend Tier	React 19, JavaScript (ES6+), Material-UI (MUI v9), TailwindCSS, Framer Motion, Swiper.js
Backend Tier	Node.js, Express.js (MVC routing, Multer for multipart form-data, Nodemailer, JSON Web Tokens)
Database Tier	MongoDB, Mongoose ODM (Object Document Mapper), Cloudinary (for file storage and hosting)
Dev & Build Tools	Vite (Fast bundling & HMR), ESLint (Linting rules), Postman (API endpoint testing), Git & GitHub

Key Implementation Decisions:

- React 19 & Vite: Selected to ensure rapid Hot Module Replacement (HMR) and minimized production bundle sizes. Vite handles module resolution efficiently and provides quick build times.
- TailwindCSS & Material-UI: Leveraged MUI for pre-built layout inputs, rating systems, and interactive buttons, while utilizing TailwindCSS for customized grid alignments, spacing variables, and page layouts.
- Cloudinary Integration: Rather than storing resume PDF uploads and article images directly on the backend filesystem (which is volatile on cloud environments), the server uploads files directly to Cloudinary and stores secured HTTPS URLs in the MongoDB database.

7. Authentication & Authorization

Access to administrative operations is secured using JSON Web Token (JWT) standards:

- Credential Verification: Admin login verifies password hashes using bcryptjs on the server.
- Token Signature: Upon verification, the server signs a JWT containing user identity variables and a cryptographic key from server configuration.
- Stateless Guard: The client stores the JWT in session storage and applies it as a bearer token inside Authorization request headers.
- Validation Middleware: A custom server middleware intercepts admin routes, verifies token signatures, and permits actions only upon token validation.

8. Responsive Design Approach

To accommodate different target users on different devices, the UI adapts dynamically using a multi-layered approach:

- Fluid Layout Grids: Material-UI's Grid layouts utilize fractional breakpoints (e.g. xs: 12, sm: 6, md: 3) to adapt rows. Testimonial sliders dynamically change slide counts based on screen width.
- Typography Scaling: Responsive classes are set on text elements to ensure readability across all resolutions, preventing text from overflowing layout boxes.
- Viewport Alignment: Elements are styled to render cleanly within screen boundaries, preventing horizontal scrollbars.

9. Error Handling

Both client and server layers incorporate defensive programming techniques to handle errors:

- Client Side Validation: Inbound input values are validated before API calls. Network failures or validation errors trigger toast alerts via Sonner.
- Server Try-Catch Blocks: Server middleware wraps endpoints in try/catch handlers. Internal errors return HTTP status codes (500) and structured error messages, preventing server crashes.
- Fallback Visuals: In the event of network connectivity drops, static fallback placeholders prevent the UI from breaking.

10. Security Measures

The project incorporates key security measures to protect database and server resources:

- Encryption: Admin passwords are saved as secure hashes using bcryptjs encryption rounds.
- Route Protection: Admin CRUD actions verify JWT headers to prevent unauthorized requests.
- CORS Configuration: Cross-Origin Resource Sharing is configured to restrict unauthorized backend access.
- Environment Variables: Sensitive connection strings, ports, and API keys are stored in a .env file, keeping credentials separate from version control.

11. Performance Optimizations

The application implements performance optimization techniques to improve page speed and performance:

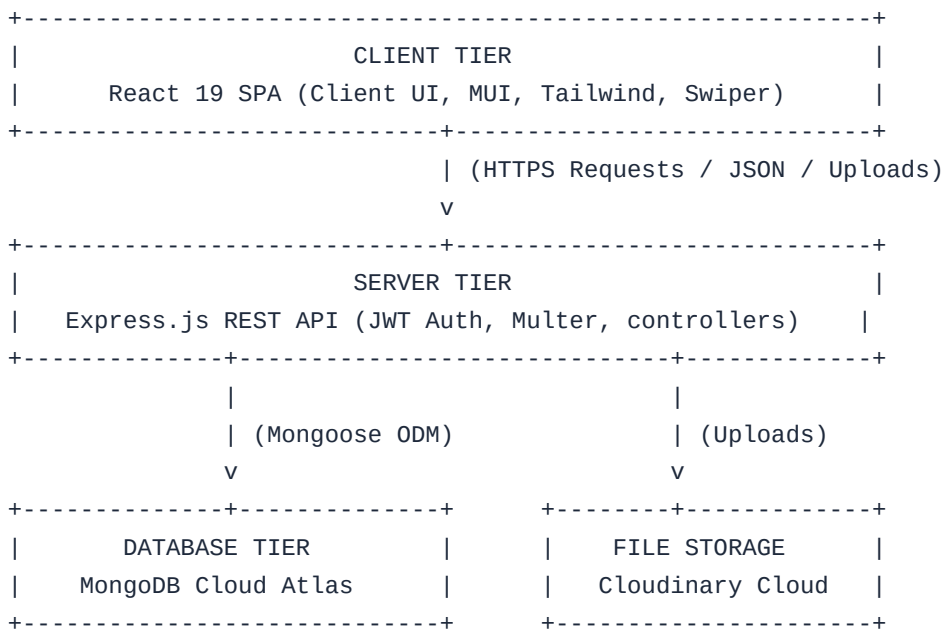
- Route Lazy Loading: React pages are lazy-loaded using Suspense fallback indicators. This splits production bundles into smaller, route-specific files, reducing initial load times.
- Asset Optimization: Content images are optimized, and document assets (like candidate CV resumes) are hosted and delivered via Cloudinary's content delivery network (CDN).
- Database Indexing: Indexes are created on frequently queried document keys (e.g. isApproved flag on reviews) to improve query execution times.

12. Git Workflow Followed During Development

The team followed a clean, structured version control workflow to coordinate project development:

- Feature Branches: Master branches remained stable. Team members worked on separate feature branches (e.g., feature/careers, feature/admin-dashboard).
- Pull Requests (PRs): Code additions were submitted through PRs, allowing for peer code reviews before merging into the main branch.
- Merge Conflict Resolution: Conflicts were resolved locally in development workspaces to maintain a stable main branch history.
- Commit Standards: Commits were structured with clear tags (e.g., feat:, fix:, refactor:) to simplify tracking.

13. System Architecture & Data Flow



Architectural Data Flow Description:

- The React client makes API calls using Axios instances to endpoints on the Express backend server.
- For public requests (testimonials list, details), the server queries MongoDB and responds with data.
- When a candidate uploads a resume, the server intercepts the multipart form-data via Multer, streams the PDF binary to Cloudinary, receives the HTTPS hosting URL, and creates a MongoDB record linking the applicant data to the resume URL.
- For administrative requests, the client attaches a JWT authorization token to request headers. The server verifies this token via custom middleware before granting CRUD privileges.

14. Project Structure

```

strivo-consultancy/
|-- client/
|   |-- public/           # Static public assets
|   |-- src/
|       |-- Admin/       # Admin pages (Dashboard, Login, ReviewsDesk, JobApplications)
|       |-- assets/      # Images and strategic illustrations
|       |-- Components/  # Reusable UI elements (Navbar, Footer, Ready CTA, SEO tags)
|       |-- pages/       # Page modules (Home, About, Services, Mission, Vision, Values,
Contact)
|       |-- services/    # API services module (allApi.js, commonApi.js, serverUrl.js)
|       |-- App.jsx      # Main Router component
|       |-- index.css    # Tailwind and typography directives
|       +-- main.jsx     # React application entry point
|   |-- package.json
|   +-- vite.config.js
+-- server/
    |-- src/
    |   |-- controllers/  # Route logic handlers (article, inquiry, career, review)
    |   |-- db/          # Database connection establishment (dbConnection.js)
    |   |-- middleware/  # JWT verification and Multer file upload handlers
    |   |-- models/      # Mongoose schema definitions (Article, JobApplication, Inquiry,
Review)
    |   |-- routes/      # Express route definitions (router.js)
    |   +-- server.js    # Main application server launch file
    |-- .env             # Server configuration (PORT, MONGO_URI, CLOUDINARY_KEYS, JWT_SECRET)
    +-- package.json

```

15. Website Pages Hub & System Catalog

The Strivo Consultancy web application consists of a total of 28 distinct page files and view modules. These are divided into 17 client-facing modules and 11 admin-facing modules. Below is a comprehensive list detailing each of these page files and their functional purposes within the system:

Page Module File	Purpose & Content Description
Home (Home.jsx)	Introduces the firm, maps core expertise, runs the client trust counters, and renders the success stories swiper.
About Us (About.jsx)	Shares Strivo's corporate background, mission/vision card triggers, leadership grid, and values cards.
Services (Services.jsx)	Primary hub detailing our key consulting categories: Strategic, Operations, Digital, and Change.
Strategic Advisory (Strategic.jsx)	Detailed consulting subpage specifying strategic roadmap planning and transformations.
Operations (Operations.jsx)	Detailed consulting subpage detailing process improvements, workflow mapping, and cost savings.
Digital Integration (Digital.jsx)	Detailed consulting subpage showing system modernization and tech integration offerings.
Change Consulting (Change.jsx)	Detailed consulting subpage focusing on operational culture transformation and management.
Our Mission (Mission.jsx)	Details our delivery methodology (Analytical Rigor, Strategic Foresight, Flawless Execution, and Growth).
Our Vision (Vision.jsx)	Presents long-term enterprise goals (Global Footprint, Integrity, Innovation, and Legacy Impact).
Core Values (Values.jsx)	Dynamic subpage rendering relevant details for Integrity, Innovation, Impact, or Collaboration.

Page Module File	Purpose & Content Description
Insights (Insight.jsx)	Renders paginated, categorized article cards allowing users to learn about industry trends.
Article Reader (Article.jsx)	Detailed blog reader interface providing full article text and related insights blocks.
Case Studies (CaseStudies.jsx)	Client portfolio page showing completed client success stories and project outcomes.
Case Study Details	Provides detailed timelines and results of specific client consulting engagements.
Contact Us (Contact.jsx)	Provides inquiry form capture, office locations, coordinates, and contact details.
Careers Portal (Career.jsx)	Presents open roles at Strivo and handles inbound candidate resume files uploads.
Reviews Submission (Review.jsx)	Feedback collection page where verified clients can submit reviews for administrative desk review.
Admin Login (Login.jsx)	Secure authentication gate verifying administrator credential hashes via bcrypt.
Admin Forgot Password (ForgotPassword.jsx)	Password recovery request interface for administration accounts.
Admin Reset Password (ResetPassword.jsx)	Verification gate for updating credentials with signature validation.
Admin Dashboard (Dashboard.jsx)	Main console shell wrapping statistics counters and sidebar navigation widgets.
Admin Careers Manager (Admincareers.jsx)	Admin view to manage job details, list open postings, and review/download applicant CVs.
Admin Articles Console (ArticlesAdmin.jsx)	Grid manager for admin to review, write, edit, and delete insights blog posts.

Page Module File	Purpose & Content Description
Admin Case Studies Hub (CaseStudies.jsx)	List view to manage and preview client success case listings.
Admin Create Case (CreateCaseStudy.jsx)	Form console to submit new client success study details with image uploads.
Admin Edit Case (EditCaseStudy.jsx)	Form to edit, update, or remove existing client success case details.
Admin Inquiries Desk (Inquiries.jsx)	Leads inbox displaying visitors' messages, emails, timestamps, and details.
Admin Profile Settings (Profile.jsx)	Account manager for administrative details and password updates.

16. Code Modules & API Routes

System Modules

The backend architecture is modular, containing four distinct functional controllers:

- Article Module: Handles insight blog posts (CRUD operations for admin, reading and pagination for clients).
- Review Module: Collects user-submitted testimonials. Admin controls review publishing by toggling approval flags.
- Inquiry Module: Processes user inquiries. Provides a direct connection point between clients and Strivo.
- Career Module: Manages recruitment data. Stores job details, handles multi-part form file uploads for CV/resumes, and lists applicants in the admin workspace.

API Integration Architecture

Communication between frontend and backend is managed via a dedicated API client using Axios. Below is a list of the main REST endpoints implemented in the system:

Method	Endpoint	Module	Functional Purpose
POST	/api/admin/login	Authenticati on	Validates admin credentials, returns JWT
GET	/api/success-stories	Reviews	Fetches approved client reviews for Swiper slider
POST	/api/reviews	Reviews	Submits a client review for administrative approval
GET	/api/all-articles	Articles	Fetches paginated insight articles (public)
POST	/api/add-article	Articles	Creates a new article (Admin - multipart image upload)
POST	/api/add-inquiry	Inquiry	Submits a client contact request
POST	/api/add-job	Careers	Submits job applications with CV PDF upload

17. Database Design

The MongoDB database structure utilizes four primary Mongoose collections. Each document is modeled with strict type definitions and default values where appropriate:

Collection	Document Schema Fields & Types
articles	title (String), category (String), description (String), imageUrl (String), content (String), createdAt (Date)
reviews	name (String), position (String), clientStories (String), imageUrl (String), isApproved (Boolean, default: false)
inquiries	name (String), email (String), message (String), createdAt (Date)
careers	name (String), email (String), position (String), resumeUrl (String)

18. Deployment Process

The application is prepared for cloud deployment using a structured pipeline:

- Build Compilation: The React frontend is compiled into a static, optimized production bundle using Vite's build pipeline (npm run build).
- Backend Routing: The Express server is configured to serve client build assets statically when running in production mode.
- Environment Provisioning: Cloud servers are configured with the required environment variables (including database URIs, Cloudinary credentials, and JWT keys).
- Database Connection: MongoDB Atlas connection strings are whitelisted to permit access from application servers.

19. Challenges Faced During Development

During development, the team resolved several technical challenges:

- Multipart File Streaming: Managing resume PDF uploads directly in server memory without writing temporary local files required configuring custom memory buffers for Multer and streaming binaries directly to Cloudinary API targets.
- Testimonial Overflow: Handling long user-submitted strings without spaces (which would break card boundaries) required adding CSS properties to wrap long inputs cleanly within the UI.
- UI Refactoring and Spacing: Adjusting section layouts to fit varying screen widths required refactoring component styling to ensure responsive typography and clean alignments.

20. Future Enhancements

Future updates planned for the Strivo Consultancy platform include:

- Client Login and Proposals: Implement secure client-specific dashboards to manage, review, and sign project proposals and track engagement advancements.
- Interactive Booking Scheduler: Integrate calendar scheduling tools to let clients book consulting consults directly on the site.
- Detailed Admin Audit Logs: Implement a tracking log system to record changes made in the admin dashboard.
- Multi-Language Localization: Add localization support to serve international enterprise clients.

21. Conclusion

The Strivo Consultancy Website project demonstrates a successful implementation of a modern, full-stack MERN application. By combining responsive styling, animations, and a secure administration panel, the platform establishes a strong online presence for the firm. The modular architecture and database design provide a reliable foundation for future updates, helping Strivo effectively connect with clients and manage business operations.